

SAM

➤ 관련 과목

 [보아즈](#)

Segment Anything

Abstract

Segment Anything (SA) 프로젝트는 **image segmentation**을 위한 새로운 **task, model, 그리고 데이터셋**을 소개하는 프로젝트이다. 이 프로젝트는 이미지 분할 분야에서 **파운데이션 모델(foundation model)** 구축을 목표로 한다.

대규모 웹 데이터셋으로 사전학습된 대형 언어 모델(LLMs)은 NLP 분야에서 혁신을 일으키고 있다. 이들은 강력한 **zero-shot / few-shot 일반화** 능력을 보여준다. 이러한 모델들은 **"foundation models"**라고 부르며 학습 시 보지 못한 새로운 태스크나 데이터 분포에도 일반화할 수 있다. 이 능력은 보통 프롬프트 엔지니어링을 통해 구현된다. 즉, 사람이 작성한 텍스트 프롬프트를 입력으로 주면, 언어 모델은 그 태스크에 맞는 유효한 텍스트 응답을 생성한다. 웹에서 수집한 방대한 텍스트 코퍼라로 대규모 학습을 하면, 이러한 모델의 제로샷/퓨샷 성능은 놀라울 정도로 강력해지고, 경우에 따라 **파인튜닝된 모델과 비슷하거나 그 이상의 성능**을 낸다.

파운데이션 모델은 컴퓨터 비전에도 적용되고 있지만, 아직은 그 범위가 제한적이다.

가장 대표적인 예는 **웹에서 수집한 이미지-텍스트 쌍을 align하는 방식**이다.

→ 모델에게 이미지와 그 이미지를 설명하는 자연어 텍스트(예: 캡션, 설명) 사이의 관계를 학습시키는 것

1. 학습 메커니즘: 대조 학습 (Contrastive Learning)

- **모델:** CLIP 이나 ALIGN 과 같은 모델이 이 방식을 사용한다.
- **원리:** 대조 학습의 기본 목적은 **이미지 인코더와 텍스트 인코더**라는 두 개의 별도 신경망을 훈련하여, 이들이 서로 다른 양식의 데이터를 **같은 임베딩 공간**에 효과적으로 배치하도록 만드는 것이다.

1.1. 긍정 쌍(Positive Pairs)의 '정렬' (Alignment)

'짝이 맞는(paired)' 이미지와 텍스트는 긍정 쌍으로 간주된다.

예시: 웹에서 수집한 이미지 I 와 그 이미지의 캡션 T .

- I : 푸른 하늘 아래 잔디밭에 서 있는 황금색 리트리버 사진
- T : "푸른 하늘 아래 잔디밭에 서 있는 커다란 황금색 개 한 마리"
 - **학습 목표:** 모델은 이미지 I 의 임베딩 v_I 와 텍스트 T 의 임베딩 t_T 를 계산한다. 대조 학습은 이 두 임베딩이 임베딩 공간에서 **서로 가깝게** 위치하도록(**정렬**, $v_I \approx t_T$) 인코더의 가중치를 조정한다. 즉, 두 인코더가 동일한 의미를 가진다고 판단한 데이터에 대해서는 높은 **유사성 점수**를 부여하도록 한다.

1. 2. 부정 쌍(Negative Pairs)의 '분리' (Separation)

대부분의 경우, 임의로 선택된 이미지와 텍스트는 서로 '짜이 맞지 않는' 부정 쌍으로 간주된다..

예시: 한 이미지 I_i 와 **다른** 이미지의 캡션 $T_j (i \neq j)$.

- I : 푸른 하늘 아래 황금색 리트리버 사진
- T' : "빨간색 스포츠카가 주차된 도시 거리의 모습"
 - **학습 목표:** 모델은 이미지 I 의 임베딩 v_I 와 캡션 T' 의 임베딩 $t_{T'}$ 가 임베딩 공간에서 **서로 멀리** 떨어지도록(**분리**) 학습한다.
즉, 관련 없는 이미지와 텍스트 쌍에는 낮은 유사성 점수를 부여합니다.

→ 대조 학습은 **임베딩 공간에서의 거리**를 조절하여 학습함.

2. 제로 샷 일반화의 기반: 통합 임베딩 공간

제로 샷 일반화의 작동 방식은 세 단계로 나눌 수 있다.

A. 학습: 의미적 정렬 (Semantic Alignment)

모델(예: CLIP)은 수억 쌍의 이미지와 그 캡션 데이터로 **대조 학습**을 수행한다.

B. 문제 정의: 텍스트 프롬프트 활용

모델이 **학습 때 보지 못한 새로운 분류 클래스** C_{new} 를 인식해야 하는 상황을 가정해보자.

- 예를 들어, 모델은 아프리카 들개라는 종을 학습 데이터에서 한 번도 보지 못했을 수 있다.
- **제로 샷**을 위해, 이 클래스의 이름을 직접적인 **분류 레이블**로 사용하는 대신, **텍스트 프롬프트**로 변환한다.
 - **프롬프트:** "A photo of an **African Wild Dog**." (아프리카 들개의 사진)

- 이 프롬프트를 텍스트 인코더에 넣어 **프롬프트 벡터**(t_{prompt})를 얻는다. 이 벡터는 모델이 학습한 의미 공간에서 "아프리카 들개"라는 개념의 의미적 위치를 나타낸다.

C. 추론: 유사도 기반 예측 (Similarity-Based Prediction)

이제 모델에게 새로운 이미지 I_{new} 가 주어지게 되.

1. **이미지 인코딩:** I_{new} 를 이미지 인코더에 넣어 이미지 벡터 v_{new} 를 얻는다.
2. **유사도 계산:** 이미지 벡터 v_{new} 와 미리 정의된 모든 후보 프롬프트 벡터(t_{prompt}) 간의 코사인 유사도(C를 계산한다.
 - **코사인 유사도:** 두 벡터가 가리키는 방향이 얼마나 유사한지를 측정하며, 임베딩 공간에서 두 개념이 얼마나 의미적으로 가까운지를 나타낸다.
3. **예측:** 계산된 유사도 중 **가장 높은 값**을 가진 프롬프트가 이미지의 예측 클래스가 된다.

3. 제로 샷의 근본적인 작동 원리

이 메커니즘이 작동하는 이유는 다음과 같다.

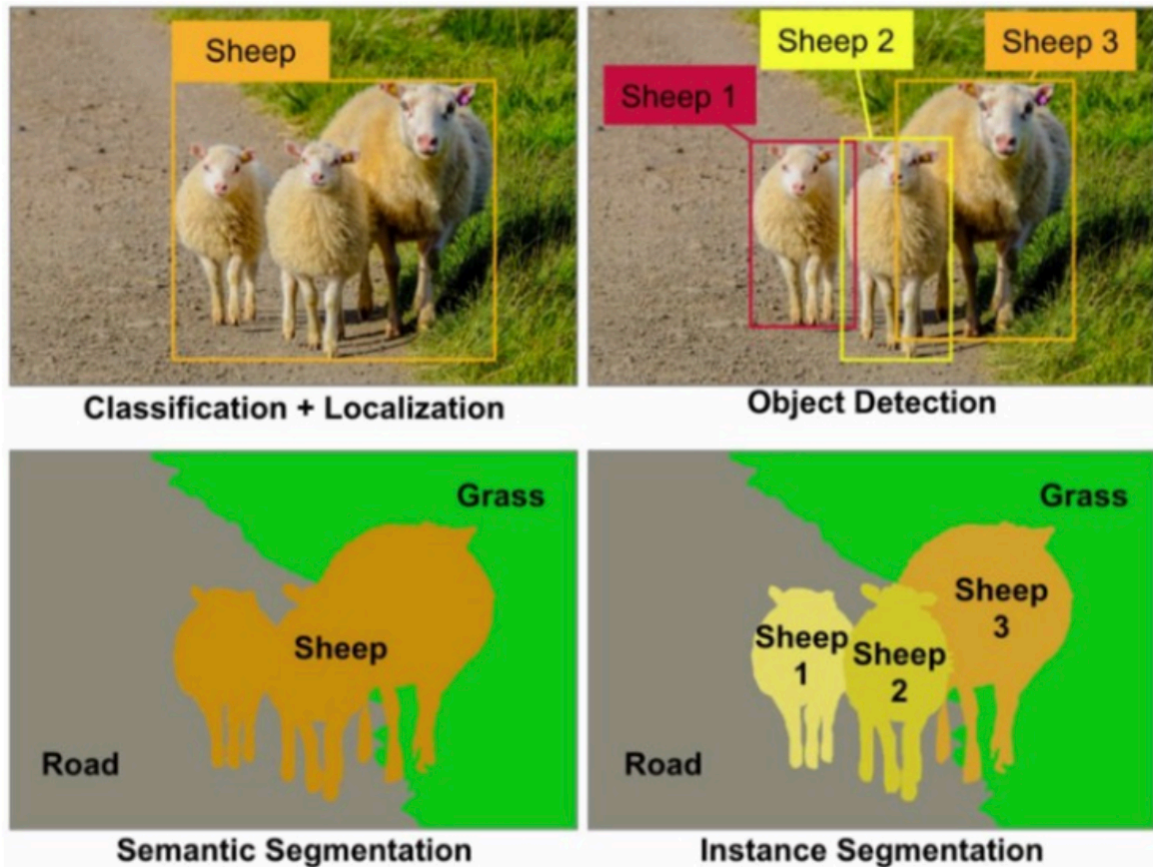
- **일반화된 의미 공간:** 모델은 "들개"라는 단어와 관련된 시각적 특징, "아프리카"와 관련된 환경적 특징 등, **개별적인 구성 요소**를 학습했다. 이 모델은 "아프리카 들개"라는 새로운 개념의 텍스트 벡터가 "개" 이미지의 벡터와 가깝고, 동시에 "늑대"나 "하이에나" 같은 유사 동물의 벡터와도 관계가 있음을 알고 있다.
- **프롬프트의 역할:** 텍스트 프롬프트는 이미지 인코더에게 "이 벡터는 이 텍스트의 의미와 같아야 한다"고 직접적으로 명령하는 인터페이스 역할을 한다. 학습을 통해 이미 시각-언어 의미가 **연속적인 공간**에 매핑되어 있기 때문에, 이 "명령"이 새로운 개념에도 효과적으로 일반화되는 것이다.

결론적으로, 대조 학습은 이미지와 텍스트를 같은 공간에 "언어적 렌즈"로 연결해 주며, 이 렌즈를 통해 모델은 학습하지 않은 시각적 대상을 텍스트 개념을 통해 이해하고 인식하는 **제로 샷 일반화**를 달성한다.

기존 Fully Connected Layer를 Fully Convolutional Layer로 바꾸면서 **pixel-level segmentation**이 가능해졌다.

- **기존 (FC layer):**
 - CNN의 마지막에 FC layer를 붙이면 → 고정된 길이의 벡터(클래스 확률)만 출력.
 - 즉, "이 이미지 = 고양이" 같은 전체 이미지 단위 분류만 가능.
- **변경 (Conv layer):**

- FC layer를 Convolution으로 바꾸면, 출력이 **2D feature map** 형태 유지.
- 그래서 픽셀 단위로 "이 위치는 고양이, 저 위치는 배경"을 예측 가능.
- → **pixel-level segmentation** 가능해짐.



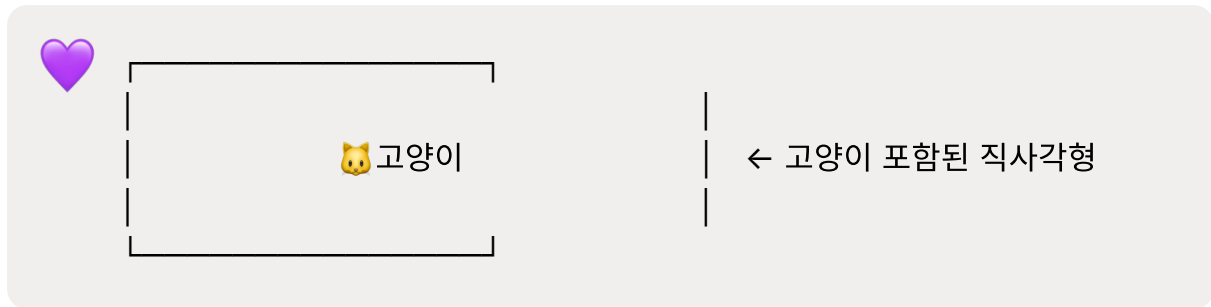
Computer vision — Object detection and segmentation

- **Classificaiton and Localization:** Object의 위치와 그 class를 예측.
- **Object Detection:** 객체를 구분하여 각각의 위치와 class를 예측.
- **Semantic Segmentation:** pixel-level로 class를 예측.
- **Instance Segmentation:** 객체를 구분하여 pixel-level로 class를 예측.

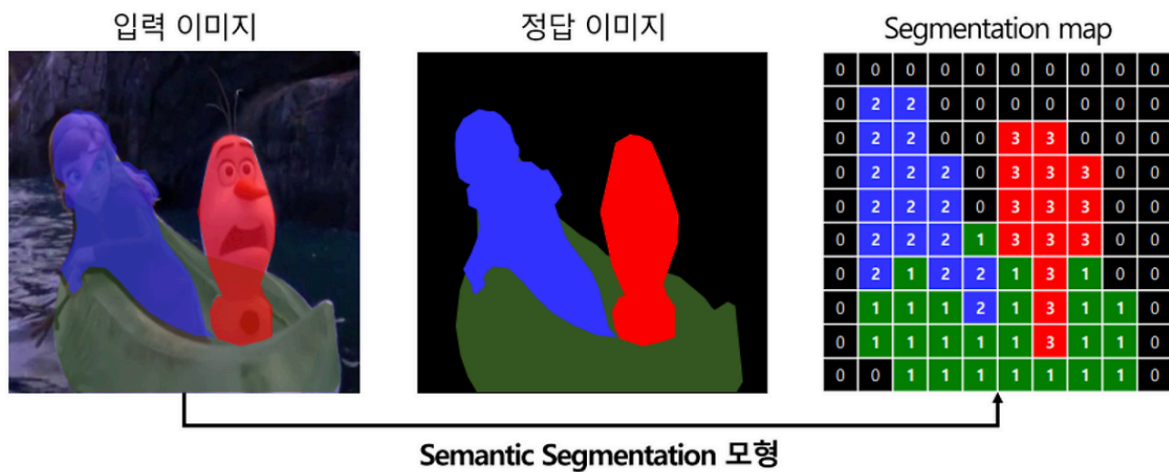
바운딩 박스 (Bounding Box)

- 사각형(x, y, w, h) 좌표로 객체 위치를 표시
- "이 사각형 안에 고양이가 있다"
- 장점: 간단하고 빠름
- 단점: 정확한 모양/경계는 표현 못 함

- 예: 고양이가 귀가 뽕족한데, 박스는 귀까지 포함한 큰 사각형으로만 잡음



마스크 (Mask)



- 픽셀 단위로 객체 영역을 채운 지도
- "고양이인 픽셀은 1, 아닌 픽셀은 0"
- 장점: 객체의 정확한 모양, 경계, 면적까지 알 수 있음
- 단점: 만들기 어렵고, 연산량이 많음

♥ 배경=0, 고양이=1

[0 0 1 1 1 0 0]

[0 1 1 1 1 1 0]

[0 0 1 1 1 0 0]

→ 실제 고양이 형태 그대로 픽셀 단위 표시

Introduction

이 모델들이 **규모가 확장되고** 웹에서 가져온 풍부한 텍스트 코퍼스로 훈련될 때, 그 제로-샷 및 소수-샷 성능은 미세 조정된 모델에 비해 놀라울 정도로 훌륭하게 (심지어 어떤 경우에는 맞먹을 정도로) 비교된다.

경험적 추세에 따르면 이러한 성능은 **모델 규모, 데이터셋 크기**, 그리고 총 훈련 계산량이 증가함에 따라 향상된다.

이 계획의 성공은 세 가지 요소, task, 모델, 데이터에 달려 있다.

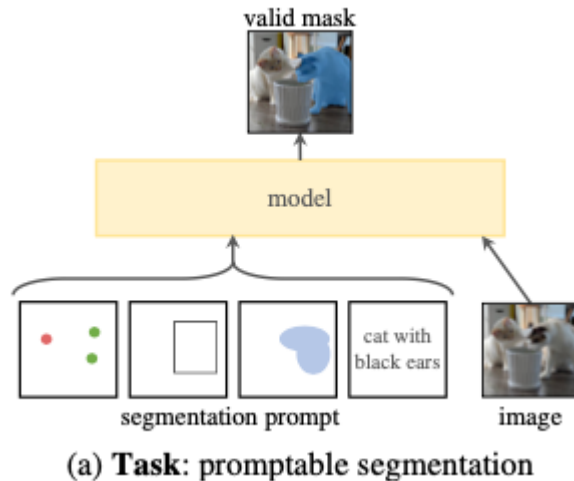
이를 개발하기 위해 image segmentation에 대한 다음 질문을 해결한다.

1. Zero-shot 일반화를 가능하게 하는 task는 무엇인가?
2. 해당 모델 아키텍처는 무엇인가?
3. 어떤 데이터가 이 task와 모델에 힘을 실어줄 수 있는가?

이러한 질문들 **복잡하게 얽혀**있어 포괄적인 해결책이 필요하다.

저자는 강력한 사전 학습 목표를 제공하고 광범위한 다운스트림 애플리케이션을 가능하게 할 만큼 충분히 일반적인 프롬프트 분할 작업을 정의하는 것에서 시작한다. 이 작업은 flexible prompting을 지원하고, 대화형 사용을 위해 프롬프트가 주어졌을 때 실시간으로 분할 마스크를 출력할 수 있는 모델을 요구한다. 웹에 텍스트나 분류 레이블은 많지만, 이미지의 모든 픽셀에 대한 정밀한 분할 마스크는 거의 존재하지 않아 대규모 데이터를 수동으로 수집하기 어렵다는 한계가 있었다. 이 문제를 해결하기 위해, 저자들은 데이터 엔진을 구축했다. 모델-인-더-루프 방식을 통해, 초기 모델이 주석 작업을 도와 데이터를 효율적으로 수집하고, 수집된 데이터로 모델을 더 좋게 개선한 후, 개선된 모델이 다시 더 많은 데이터를 수집하도록 하는 **순환적인 시스템**입니다.

Task



자연어 처리와 더 최근에는 컴퓨터 비전 분야에서, 파운데이션 모델은 종종 프롬프팅 기법을 사용하여 새로운 데이터셋과 작업에 대해 제로-샷 및 소수-샷 학습을 수행할 수 있는 유망한 발전이다.

이러한 연구 흐름에서 영감을 받아, 저자는 프롬프트 분할 작업을 제안한다. 이 작업의 목표는 어떤 분할 프롬프트가 주어지더라도 유효한 분할 마스크를 반환하는 것이다. 프롬프트는 간단히 이미지에서 무엇을 분할해야 하는지 지정한다. 예를 들어, 프롬프트는 객체를 식별하는 공간적 정보나 텍스트 정보를 포함할 수 있다. 프롬프트의 범위를 점, 상자와 같은 공간적 입력부터 텍스트 설명까지 넓게 정의한다. 이는 모델이 다양한 형태의 사용자 입력을 받아들일 수 있어야 함을 의미한다. 유효한 출력 마스크에 대한 요구 사항은 프롬프트가 모호하여 여러 객체를 참조할 수 있는 경우(예: 셔츠 위의 한 점이 셔츠나 혹은 그것을 입은 사람 전체를 나타낼 수 있음)에도, 출력은 적어도 그 객체들 중 하나에 대한 합리적인 마스크여야 한다는 것을 의미한다.

→ 즉, 프롬프트 분할 작업의 핵심은 모델이 어떤 프롬프트를 받더라도 '쓸모 있는' 결과를 내놓아야 한다는 것.

이는 모호성 인식의 필요성을 설명하는데 모델은 '셔츠 위의 점'이라는 모호한 입력이 들어왔을 때, 셔츠 마스크와 사람 마스크 중 둘 다 또는 하나를 정확히 예측해야 하며, 이것이 '유효한' 마스크의 정의가 된다. 이는 언어 모델이 모호한 질문에 대해 일관성 있는 응답을 내놓는 것을 기대하는 것과 유사합니다.

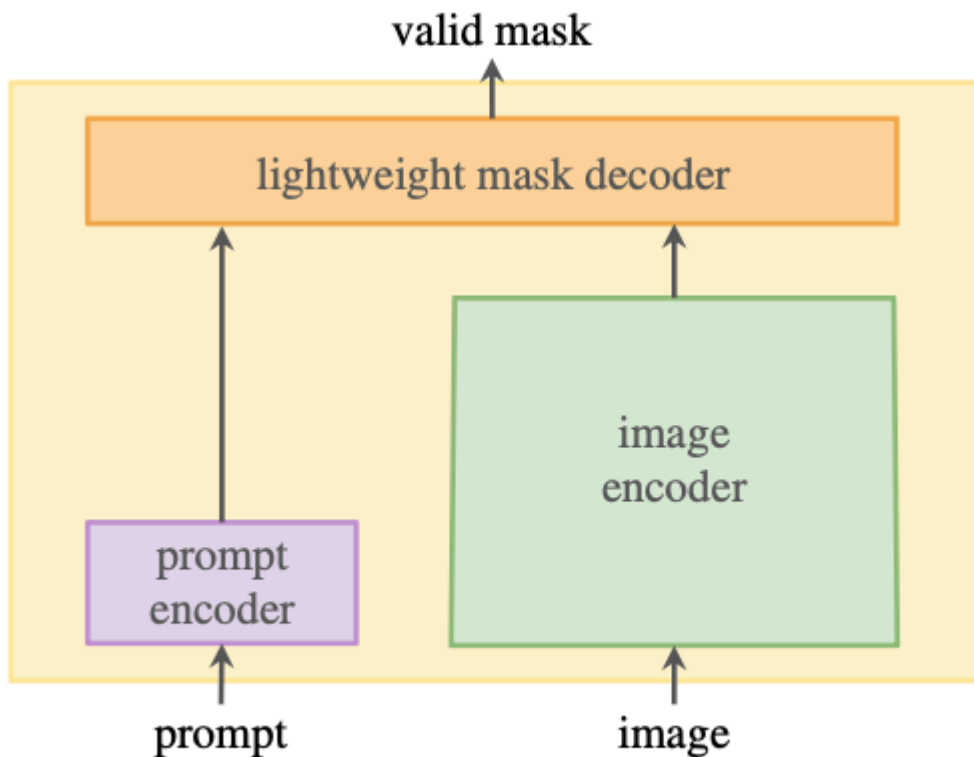
이 원리는 **언어 모델이 모호한 질문에 대해 일관성 있는 응답을 내놓는 것을 기대하는 것과 같다.**

- **언어 모델 예시:** 내가 언어 모델에게 "**bank**에 대해 설명해 줘"라고 물었을 때, 'bank'가 금융 기관인지, 강가인지 모호하다.
 - **합리적인 응답:** 언어 모델은 "금융 기관으로서의 은행"에 대해 자세히 설명하거나, "이 단어는 은행과 제방 두 가지 의미가 있다"고 답할 것이다. 절대 은행과 제방이

쉬운 이상한 개념을 출력하지 않는다.

마찬가지로, SA 모델도 모호한 입력이 들어왔을 때, "모호함에도 불구하고, 내가 인식하는 합리적인 객체 중 하나(또는 여러 개)를 정확히 분할해 주겠다"고 약속하는 것이다. 이것이 **프롬프트 분할 작업**이 요구하는 **모호성 인식** 능력이다.

Model



(b) Model: Segment Anything Model (SAM)

SAM은 단순히 연구용이 아니라 실제로 쓸 수 있는 프롬프트 기반 세그멘테이션 모델을 목표로 하기 때문에, 모델을 설계할 때 크기, 속도, 입력 처리 방식 등에 여러 제약 조건이 생긴다.

특히, 모델은 유연한 프롬프트를 지원해야 하며, 대화형 사용을 위해 **상각된 실시간으로 마스크를 계산**할 수 있어야 하고, **모호성 인식** 능력을 갖춰야 한다. 모델의 총 비용이 여러 작업에 걸쳐 분산되어 실질적으로 체감하는 속도는 실시간처럼 느껴지는 것을 의미한다. 특히 SAM에서는 **이미지 인코딩**의 무거운 계산 비용을 **한 번만** 하고, **프롬프트 처리**는 **반복적으로 빠르게** 하여 실시간 효과를 낸다. 저자는 이 **간단한 설계**가 이 세 가지 제약(유연성, 실시간, 모호성 인식)을 모두 충족한다는 것을 발견했다: 강력한 이미지 인코더는 이미지 임베딩을 계산

하고, 프롬프트 인코더는 프롬프트를 임베딩하며, 그 후 이 두 정보는 경량 마스크 디코더에서 결합되어 분할 마스크를 예측한다.

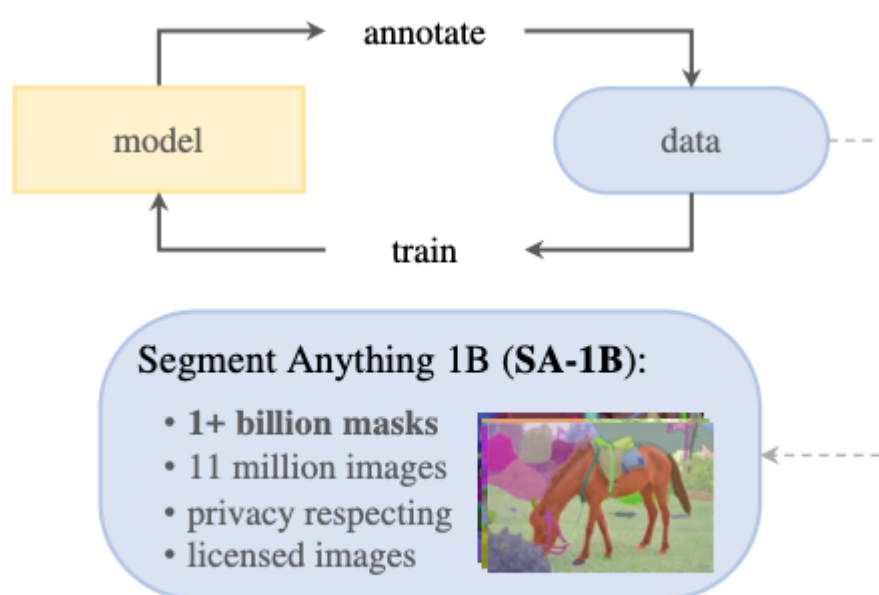
이 모델을 **Segment Anything Model**, 줄여서 **SAM**이라고 부른다. **이미지 임베딩**이 주어진다면, **프롬프트 인코더**와 **마스크 디코더**는 웹 브라우저에서 **~50ms(밀리초)** 만에 프롬프트로부터 마스크를 예측한다.

→ GPU 환경에서 돌아가는 것이 아닌 웹 브라우저에서도 돌아간다는 의미. → 경량 모델.

저자는 **점**, **상자**, 그리고 **마스크 프롬프트**에 중점을 두며, 자유 형식 텍스트 프롬프트에 대한 초기 결과도 제시한다.

저자는 SAM이 **하나의 프롬프트**에 대해 **여러 개의 마스크**를 예측하도록 설계했으며, 이는 **셔츠 대(vs.) 사람** 예시와 같은 모호성을 SAM이 자연스럽게 처리할 수 있게 한다.

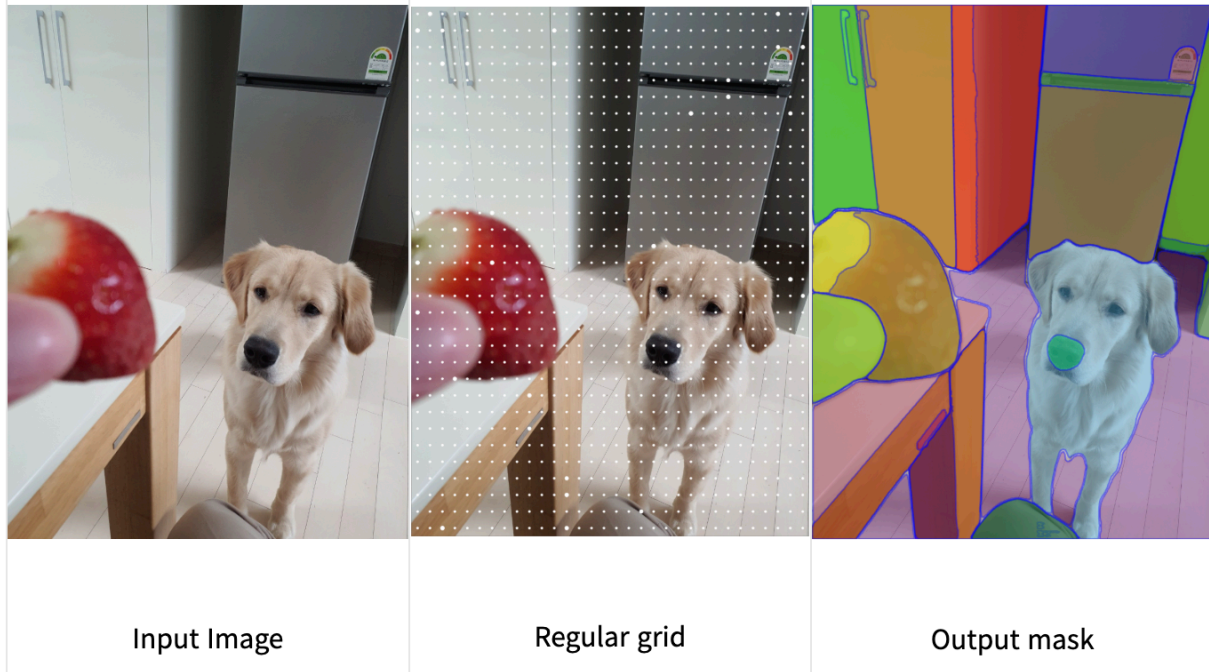
Data Engine



(c) **Data**: data engine (top) & dataset (bottom)

파운데이션 모델의 핵심인 '일반화 능력'을 얻으려면 기존의 분할 데이터셋(COCO, LVIS 등)의 규모와 다양성이 불충분했기 때문에, 새로운 대규모 데이터셋 구축이 필요했다. CLIP 같은 시각-언어 모델은 웹에 풍부한 이미지-캡션 쌍을 활용할 수 있지만, 이미지 분할 마스크(픽셀 단위의 경계 정보)는 웹에 자연적으로 대량 존재하지 않는다는 분할 분야의 고유한 데이터 한계가 있었다.

데이터 엔진은 세 단계를 거친다: 보조-수동(assisted-manual), 반자동(semi-automatic), 그리고 완전 자동(fully automatic) 단계.



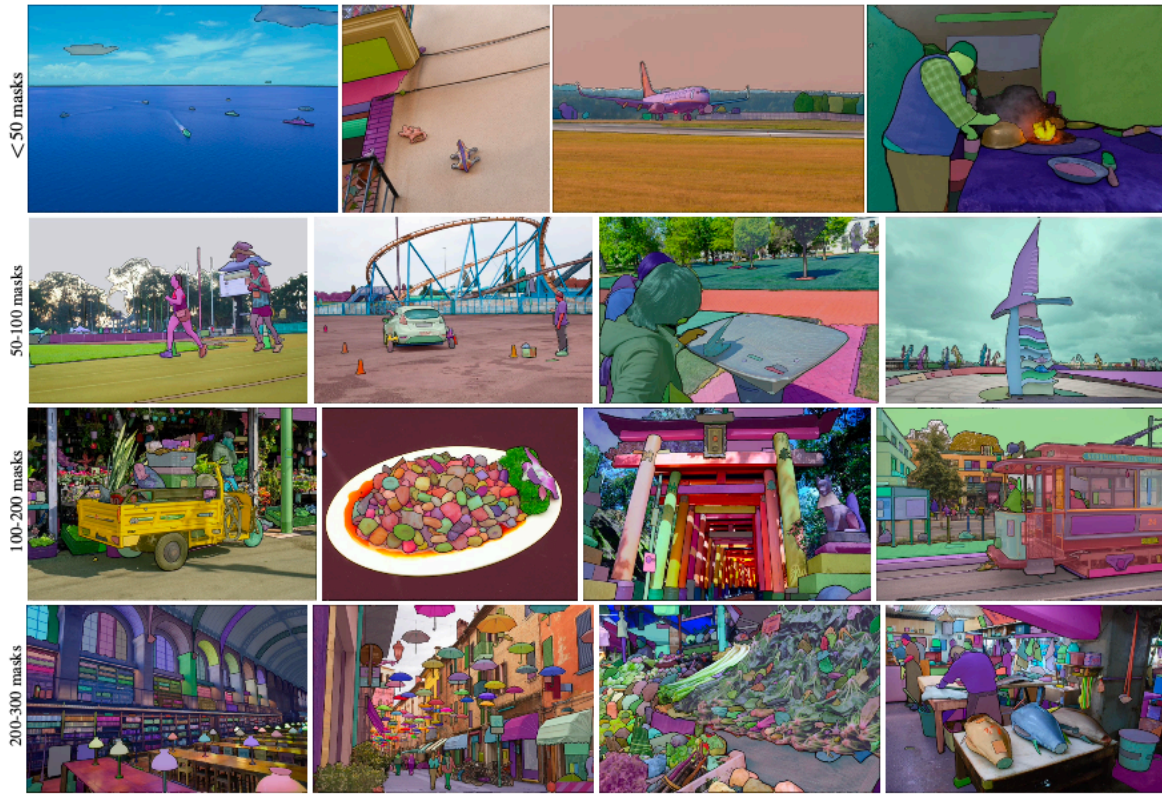
보조 수동 단계에서 사람이 점이나 상자를 찍으면 SAM이 자동으로 경계를 제안., 사람이 처음부터 픽셀을 일일이 칠할 필요 없이 오류만 수정하게 함으로써 주석 속도를 높입니다.

반작용 단계에서는 SAM은 **객체 위치를 예측**하는 프롬프트를 사용하여 **일부 객체**에 대한 마스크를 **자동으로 생성**할 수 있었고, 주석가들은 SAM이 못 잡은 **남은 객체**에 대한 주석에 집중함으로써 **마스크 다양성**을 높이는 데 도움을 주었다.

즉, 모델의 성능이 향상되자, 주석가들은 모델이 놓치거나 자신이 없는 '덜 일반적인' 객체에 집중하게 되면서 데이터셋의 다양성(무엇이든 분할할 수 있는 능력)이 커졌다.

최종 단계에서는, 규칙적인 전경 점 격자로 SAM에게 프롬프트를 제공하여, **이미지당 평균 ~100개의 고품질 마스크**를 생성했다. 이 단계에서 SA-1B의 10억 개가 넘는 마스크 대부분이 생성되었다.

Dataset



최종 데이터셋인 **SA-1B**는 1천 1백만 개(11M)의 라이선스를 받은 및 개인 정보 보호 이미지로부터 **10억 개(1B) 이상의 마스크**를 포함한다. 데이터 엔진의 최종 단계를 사용하여 완전히 자동으로 수집된 SA-1B는 기존의 어떤 분할 데이터셋보다도 **400배 더 많은 마스크**를 가지고 있으며, 저희가 광범위하게 검증했듯이, 이 마스크들은 **고품질이며 다양성**을 갖추고 있다.

Responsible AI

저자는 **SA-1B**와 **SAM**을 사용할 때 발생할 수 있는 잠재적인 공정성 우려와 편향에 대해 연구하고 보고한다. SA-1B 데이터셋이 이미지의 출처 국가를 기준으로 볼 때 광범위한 지역과 다양한 경제 수준의 국가를 포함하고 있어 데이터의 편향이 비교적 적음을 시사한다. 그 결과, SAM 모델 역시 성별, 연령, 피부 톤 등 다양한 그룹의 사람들을 분할할 때 성능 차이가 크지 않다는 긍정적인 평가 결과를 제시한다.

Experiments

모델의 성능과 일반화 능력을 다방면으로 입증하기 위해 단순히 한두 가지 테스트에 그치지 않고 많은 실험을 수행했다. 첫째, 23개의 분할 데이터셋으로 이루어진 다양하고 새로운 스

위트를 사용하여, SAM이 단일 전경 점만으로 **고품질 마스크**를 생성하며, 이는 수동으로 주석된 정답보다 **약간만 낮은 수준**이라는 것을 발견했다. 즉, 일반화 능력을 테스트하기 위해 기존의 일반적인 데이터셋을 넘어선 **다양한 영역(예: 의료, 수중, X-ray 등)의 데이터셋**을 모아 평가 벤치마크를 만들었다. 두번째로 저자는 **프롬프트 엔지니어링**을 사용하는 **제로-샷 전송 프로토콜** 하에서, 엣지 검출, 객체 제안 생성, 인스턴스 분할, 텍스트-투-마스크 예측의 예비 탐색을 포함한 **다양한 다운스트림 작업**에서 **일관되게 강력한 정량적 및 정성적 결과**를 발견했다.

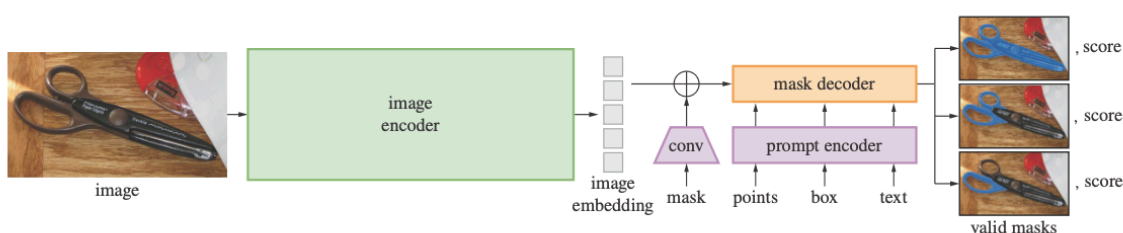
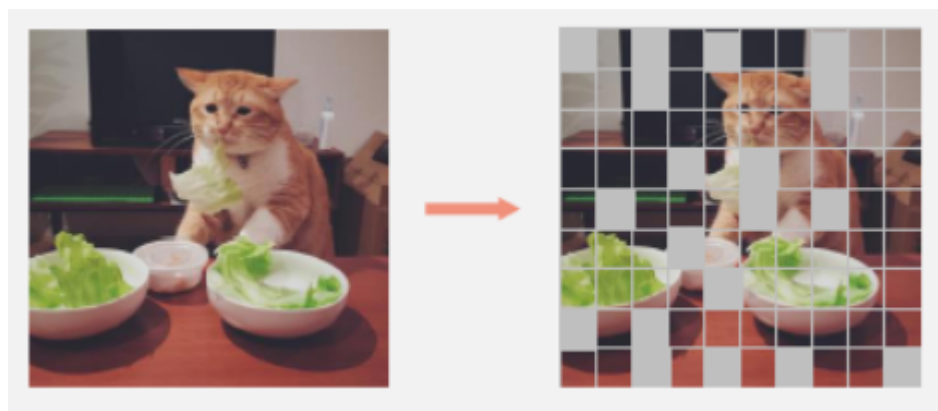


Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

Segment Anything Model

Image Encoder



확장성과 강력한 사전 학습 방법에 동기 부여되어, 저자는 MAE로 사전 학습된 ViT를 사용하며, 고해상도 입력을 처리하도록 최소한으로만 조정했다. 이미지 인코더는 이미지당 한 번

실행되며, 모델에 프롬프트를 입력하기 전에 적용될 수 있다. 이미지 인코더는 ViT 기반의 강력한 특징 추출기로, 한 번의 실행으로 이미지의 모든 시각적 정보를 벡터로 압축한다.

Prompt Encoder

SAM은 프롬프트(힌트)를 입력받아 segmentation을 함.

프롬프트는 **형태가 다 다름**:

- **희소(Sparse)**: 점, 박스, 텍스트
- **밀집(Dense)**: 마스크

그래서 Prompt Encoder는 이런 다양한 프롬프트들을 **하나의 공통 벡터 표현**으로 바꿔줘야 함.

→ 그래야 마스크 디코더가 일관되게 쓸 수 있음.

1. 희소 프롬프트 (Sparse Prompts)

(1) 점(Point)

- 예: 사용자가 고양이 귀 위를 클릭함.
- 이 점은 “고양이가 여기 있음”이라는 힌트.
- 하지만 점은 그냥 (x, y) 좌표일 뿐 → 신경망이 바로 이해하기 어려움.
- 그래서 Positional Encoding을 써서 좌표를 고차원 벡터로 바꿈.
- 거기에 “이건 점이다”라는 학습된 토큰(embedding)을 더해줌.

→ “고양이 귀 좌표 + 점이라는 의미”가 담긴 벡터

(2) 박스(Box)

- 예: 사람이 강아지를 둘러싸는 네모 박스를 그림.
- 박스 좌표 (x_min, y_min, x_max, y_max)를 **위치 인코딩**으로 바꿔 벡터화.
- 여기에 “**이건 박스 프롬프트다**”라는 토큰을 더함.

→ “강아지가 이 박스 안에 있다”는 정보가 벡터로 변환됨.

(3) 텍스트(Text)

- 예: 사람이 “cat”이라고 입력.
- SAM은 **CLIP의 텍스트 인코더**를 가져다 씀.
- “cat” → CLIP이 만든 텍스트 임베딩 → 이미지와 잘 정렬된 벡터 표현.

→ “고양이라는 개념”이 벡터로 변환됨.

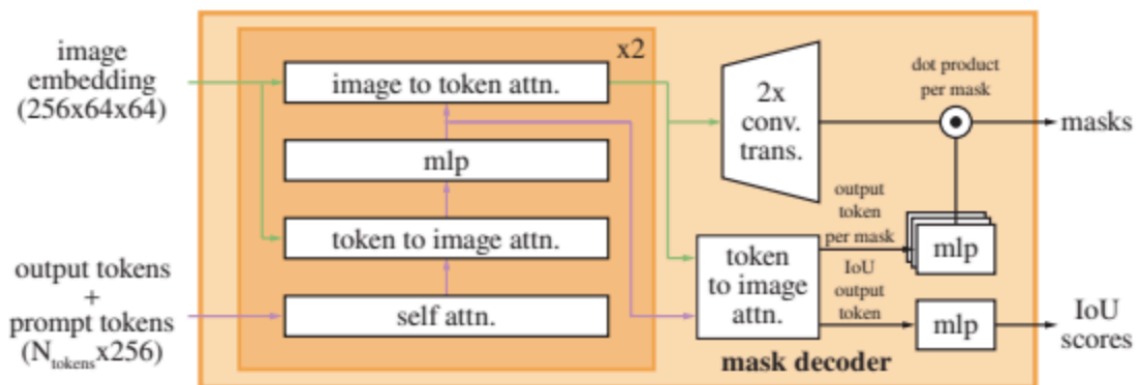
2. 밀집 프롬프트 (Dense Prompts)

마스크(Mask)

- 예: 사람이 대충 색칠한 영역(고양이 몸통 부분)을 프롬프트로 제공.
- 이걸 픽셀 단위 정보가 들어있음 → 공간적으로 풍부한 입력.
- SAM은 이 마스크를 컨볼루션으로 인코딩해서 벡터 맵으로 변환.
- 그리고 이 마스크 벡터를 이미지 임베딩과 element-wise sum(요소별 덧셈)으로 합쳐 줌.

→ 이미지 특징 맵 안에 “여기가 관심 있는 영역”이라는 신호가 추가됨.

Mask Decoder



마스크 디코더는 이미지 임베딩, 프롬프트 임베딩, 그리고 출력 토큰을 효율적으로 마스크에 매핑한다.

이 설계는 트랜스포머 디코더 블록을 수정한 형태와 그 뒤에 동적 마스크 예측 헤드를 사용한다. 수정된 디코더 블록은 프롬프트 자체-어텐션과 두 방향의 교차-어텐션을 사용하여 모든 임베딩을 업데이트한다 (프롬프트-대-이미지 임베딩 및 그 반대). 두 블록을 실행한 후, 이미지 임베딩을 업샘플링하고 MLP가 출력 토큰을 동적 선형 분류기에 매핑하여 각 이미지 위치에서 마스크 전경 확률을 계산한다.

Transformer 디코더 블록

- 원래 Transformer는 **Query-Key-Value Attention**으로 입력 간 관계를 학습한다.

- SAM에서는 이걸 살짝 바꿔서:
 - **프롬프트끼리만 서로 보는(Self-Attention)**
 - 점, 박스, 텍스트 힌트들 사이에서 정보 교환
 - **프롬프트 ↔ 이미지 특징 서로 보기 (Cross-Attention, 양방향)**
 - 프롬프트가 "이미지의 어디를 봐야 할지" 알려주고
 - 이미지도 "이 부분이 프롬프트랑 관련 있다"라고 응답

업샘플링 (Upsampling)

- Transformer는 특징 맵 크기를 줄여서 계산하기 때문에, 원래 이미지보다 해상도가 낮음.
- 그래서 다시 해상도를 키워(patch 크기 복원) 원래 이미지 크기와 맞춰줌.

Dynamic Linear Classifier (동적 분류기)

- 일반 세그멘테이션 모델은 "고정된 분류기"를 써서 픽셀마다 클래스(고양이/배경)를 판별.
- SAM은 **동적 분류기**를 씀:
 - 프롬프트(예: "고양이")에 따라 **분류기의 가중치가 바뀜**
 - 그래서 같은 이미지라도 프롬프트가 "cat"이면 고양이 마스크, "dog"이면 강아지 마스크 출력

최종 출력

각 픽셀에 대해 "전경일 확률"을 계산, 확률 맵(heatmap)을 마스크로 변환

→ 사람이 원하는 객체 부분만 정확히 색칠된 segmentation mask 완성



Figure 3: Each column shows 3 valid masks generated by SAM from a single ambiguous point prompt (green circle).

3개의 마스크 출력이 가장 일반적인 경우(전체 영역, 부분 영역, 하위 영역: 중첩 마스크는 보통 최대 세 단계 깊이임)를 다루기에 충분함을 발견했다. 훈련 중에는 마스크에 대한 **최소 손실**만 역전파합니다. 마스크 순위를 매기기 위해, 모델은 각 마스크에 대한 신뢰도 점수(예상 IoU)를 예측한다.

일반적인 세그멘테이션 학습

- 보통 모델은 **입력 → 출력 마스크 1개**
- 이 마스크를 정답과 비교해서 손실을 계산
- 그 손실을 줄이는 방향으로 학습

즉, "예측 1개 vs 정답 1개" 구조로 단순함.

SAM은 조금 다름

→ SAM은 **프롬프트 하나**에 대해 **마스크 후보를 여러 개(보통 3개)** 만들어냄.

예시: 사용자가 고양이 귀에 점을 찍음

→ SAM 출력:

1. 마스크 A: 고양이 전체
2. 마스크 B: 고양이 귀 부분
3. 마스크 C: 귀 + 주변 배경

여기서 학습 데이터의 정답 마스크는 **고양이 전체**라고 할 때.

SAM이 학습하는 방법

- SAM은 이 3개 후보(A, B, C)를 **정답 마스크와 각각 비교**해서 손실을 계산함.
 - A vs 정답 → 손실이 작음 (거의 맞음)
 - B vs 정답 → 손실이 큼 (부분만 맞음)
 - C vs 정답 → 손실이 더 큼 (틀림 많음)
- SAM은 이 중에서 손실이 가장 작은 것(= 정답과 제일 비슷한 후보)만 골라서 역전파에 사용한다.
 - 즉, A가 정답에 제일 가까우면 A만 가지고 가중치를 업데이트.
 - B, C는 이번 학습 업데이트에 쓰이지 않음.

왜?

SAM이 후보를 여러 개 내놓는 이유는 **애매한 프롬프트(점 하나 등) → 정답이 여러 해석 가능하기** 때문임.

- 그러나 만약 모든 후보에 대해 강제로 학습하면 → 모델이 혼란스러워짐 (정답은 고양이 전체인데, "귀만 정답"이라고도 배우게 됨).
- 그래서 "가장 잘 맞은 후보만 정답으로 학습"하는 전략을 씬.

Mask Prediction

마스크 예측은 포컬 손실(focal loss)과 다이스 손실(dice loss)의 선형 조합으로 학습시켰다.

- **Focal Loss** → “작은 거나 어려운 부분도 잘 맞추는 것”
 - 예: 사진 속 고양이 눈동자 같은 작은 부분도 놓치지 않는 것
- **Dice Loss** → “전체 모양을 비슷하게 맞추는 것”
 - 예: 고양이 귀, 꼬리까지 포함해서 전체 실루엣이 정답이랑 잘 겹쳤는지

두 개를 합쳐서 → 픽셀 단위 정확도 + 마스크 전체 모양 일치를 동시에 잡아냄

프롬프트 가능한 세그멘테이션 과제를 학습할 때, 다양한 기하학적 프롬프트(점, 박스, 마스크)를 섞어서 사용했다.

- 즉, 학습 때 사용자가 점만 찍을 수도 있고, 박스를 그릴 수도 있고, 마스크를 줄 수도 있는 상황을 모두 시뮬레이션함.

마스크당 11라운드의 프롬프트를 무작위로 샘플링해서 대화형 환경을 시뮬레이션했다.

- **11라운드 시뮬레이션:**
 - 예: 한 이미지에 고양이 마스크가 정답이면,
 - 1라운드: 귀 위에 점 찍기
 - 2라운드: 박스로 고양이 둘러싸기
 - ... 이런 식으로 총 11번 다른 방식 프롬프트를 제공
 - 실제 사용 환경과 유사하게 훈련되어 데이터 엔진과 통합됐음.